FIT3155
Advanced algorithms and data structures

Week 1 lecture notes by Taylor Kearney

Based on lecture slides by Arun Konagurthu

MONASH University

Faculty of Information Technology

September 11, 2025

Contents

1 Linear-time exact pattern matching . . . 2
2 Naive pattern matching . . . 2
3 Improving on naive pattern matching . . . 5
4 Gusfield's Z-Algorithm . . . 6
4.1 The Algorithm . . . 9
4.2 Summary . . . 14
4.3 Application to the exact pattern matching problem . . . 15

1 Linear-time exact pattern matching

Sources and recommended reading: Dan Gusfield, Algorithms on Strings, Trees and Sequences, Cambridge Press. (Chapters 1-2).

We begin with an examination of a problem that is ubiquitous in modern computing; the *exact pattern matching problem*.

Definition 1: The exact pattern matching problem

Given a string **txt** $[1...n]$ called the *reference text* and a second string **pat** $[1...m]$ called the *pattern*, find all occurrences, if any, of **pat** in **txt**.

Example 1: Exact pattern matching

If we have, $\mathbf{txt}[1\ldots12] = \overset{1}{\mathtt{b}}\,\overset{2}{\mathtt{b}}\,\overset{3}{\mathtt{a}}\,\overset{4}{\mathtt{b}}\,\overset{5}{\mathtt{a}}\,\overset{6}{\mathtt{x}}\,\overset{7}{\mathtt{a}}\,\overset{8}{\mathtt{b}}\,\overset{9}{\mathtt{a}}\,\overset{10}{\mathtt{b}}\,\overset{11}{\mathtt{a}}\,\overset{12}{\mathtt{y}}$ and $\mathbf{pat}[1\ldots3] = \overset{1}{\mathtt{a}}\,\overset{2}{\mathtt{b}}\,\overset{3}{\mathtt{a}}$ then **pat** occurs in **txt** at positions 3,7 and 9, as shown by the alignments below,

	1	2	3	4	5	6	7	8	9	10	11	12
txt:	b	b	a	b	a	x	a	b	a	b	a	y
pat:			a	b	a							
			✓	✓	✓							
pat:							a	b	a			
							✓	✓	✓			
pat:									a	b	a	
									✓	✓	✓	

The practical importance of such a problem should be obvious, and arises in a variety of domains including, but by no means limited to:

- Internet search engines such as Google.
- Word processors; think about the grep command in Unix or 'find in file' (crtl+f) command in Microsoft Word (and other applications).
- Library catalogues.

2 Naive pattern matching

It is often instructive, and somewhat traditional to begin by considering the naive solution to a problem before proceeding to more ingenious solutions. A naive approach to exact pattern matching would begin by aligning the left end of **pat** with the left end of **txt** and performing character comparisons left to right. In the first alignment it will compare **pat**[1] with **txt**[1], **pat**[2] with **txt**[2], and so on, until either a mismatch is found,

$$\mathbf{pat}[i] \neq \mathbf{txt}[i], \text{ for } \textbf{some } 1 \leq i \leq m,$$

or a complete match occurs,

$$\mathbf{pat}[i] = \mathbf{txt}[i], \text{ for } \textbf{all } 1 \leq i \leq m,$$

In either case, we proceed by shifting **pat** one place to the right so that **pat**[1] is now aligned with **txt**[2], and character by character comparisons are restarted from the left end of **pat**. This continues until we exhaust all the valid alignments between **pat** and **txt** (the right end of **pat** moves past the right end of **txt**). If at any point we find a complete match we report it by simply printing the index in **txt** that marks the start of the current alignment. This naive algorithm is detailed more succinctly by the pseudocode below.

```
n = |txt|
m = |pat|
for i from 1 to n-m+1  do
    for j from 1 to m do
        if txt[i+j-1] != pat[j] then
            break  // mismatch
        endif
    endfor;
    if (j == m+1) print i;
endfor
```

Example 2: Naive pattern matching

To investigate the performance of our naive algorithm let's consider, $\mathbf{txt}[1...10] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{a}}\ \overset{3}{\mathtt{a}}\ \overset{4}{\mathtt{a}}\ \overset{5}{\mathtt{a}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\mathtt{a}}\ \overset{8}{\mathtt{a}}\ \overset{9}{\mathtt{a}}\ \overset{10}{\mathtt{a}}$ and $\mathbf{pat}[1...3] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{a}}\ \overset{3}{\mathtt{a}}$. The algorithm will explore the following alignments, where the iteration is given on the far left,

	1	2	3	4	5	6	7	8	9	10
txt:	a	a	a	a	a	a	a	a	a	a
pat:	a	a	a							
iteration-1	✓	✓	✓							
pat:		a	a	a						
iteration-2		✓	✓	✓						
pat:			a	a	a					
iteration-3			✓	✓	✓					
⋮					⋮					
pat:								a	a	a
iteration-8								✓	✓	✓

In each alignment we find an occurrence of **pat** and so must perform 3 comparisons every iteration giving $8 \times 3 = 24$ comparisons in total.

Time complexity: Naive pattern matching

The previous example actually illustrates the worst case for our naive pattern matching algorithm. As each alignment is a complete match our comparisons never terminate prematurely and we perform a comparison for every character in **pat** in each iteration.

Let n and m denote the length of **txt** and **pat** respectively. In general, the naive method checks every valid alignment between **pat** and **txt** and in the worst case we perform m comparisons for each alignment. Therefore the total number of comparisons in the worst case is given by,

$$(n - m + 1) \times m = O(mn).$$

Using more sophisticated methods it is possible to reduce the worst case complexity to $O(m+n)$ (a significant improvement; try $m = 1000$ and $n = 10,000,000$), and our naive method is generally too slow for large texts and patterns.

3 Improving on naive pattern matching

To improve on the naive method we need to reduce the number of explicit comparisons we perform. One way of doing this is to shift **pat** right by more than a single character when a mismatch occurs. This reduces the number of alignments for which we perform explicit comparisons, consequentially reducing the number of explicit comparisons performed. However, we must ensure that we **never** shift so far as to miss an occurrence of **pat** in **txt**.

Example 3: Improving on the naive pattern matching

To better visualise this principle let's consider the example below where we wish to find all occurrences of $\mathbf{pat}[1...8] = \overset{1}{\mathtt{a}}\ \overset{2}{\mathtt{b}}\ \overset{3}{\mathtt{x}}\ \overset{4}{\mathtt{y}}\ \overset{5}{\mathtt{a}}\ \overset{6}{\mathtt{b}}\ \overset{7}{\mathtt{x}}\ \overset{8}{\mathtt{z}}$ in $\mathbf{txt}[1...13] = \overset{1}{\mathtt{x}}\ \overset{2}{\mathtt{a}}\ \overset{3}{\mathtt{b}}\ \overset{4}{\mathtt{x}}\ \overset{5}{\mathtt{y}}\ \overset{6}{\mathtt{a}}\ \overset{7}{\mathtt{b}}\ \overset{8}{\mathtt{x}}\ \overset{9}{\mathtt{y}}\ \overset{10}{\mathtt{a}}\ \overset{11}{\mathtt{b}}\ \overset{12}{\mathtt{x}}\ \overset{13}{\mathtt{z}}$. Let's start with the naive method,

	1	2	3	4	5	6	7	8	9	10	11	12	13
txt:	x	a	b	x	y	a	b	x	y	a	b	x	z
pat:	a	b	x	y	a	b	x	z					
iteration-1	✗												
pat:		a	b	x	y	a	b	x	z				
iteration-2		✓	✓	✓	✓	✓	✓	✓	✗				
pat:			a	b	x	y	a	b	x	z			
iteration-3			✗										
pat:				a	b	x	y	a	b	x	z		
iteration-4				✗									
pat:					a	b	x	y	a	b	x	z	
iteration-5					✗								
pat:						a	b	x	y	a	b	x	z
iteration-6						✓	✓	✓	✓	✓	✓	✓	✓

Overall the naive method performs 20 comparisons in this example and examines every valid alignment between the pattern and the text. However, we can see that it is not necessary to examine some of these alignments.

In iteration 2 we find that,

$$\mathbf{pat}[1...7] = \mathbf{txt}[2...8],$$

before **pat**[8] ('z') mismatches with **txt**[9] ('y'). The next three iterations $(3-5)$ then terminate after a single comparison as **pat**[1] ('a') mismatches with **txt**[3] ('b'), **txt**[4] ('x') and **txt**[5] ('y') respectively. This occurs because the next occurrence of **pat**[1] ('a') in **txt** is at position 6, and so no matches are possible to the left of this position. A smarter algorithm could use this information to perform a larger shift and immediately align **pat**[1] with **txt**[6] after iteration 2. Such a shift would avoid these unnecessary comparisons and is guaranteed to never miss an occurrence of **pat**. As shown below, with our smarter shift we save 3 comparisons, performing 17 in total.

	1	2	3	4	5	6	7	8	9	10	11	12	13
txt:	x	a	b	x	y	a	b	x	y	a	b	x	z
pat:	a	b	x	y	a	b	x	z					
iteration-1	✗												
pat:		a	b	x	y	a	b	x	z				
iteration-2		✓	✓	✓	✓	✓	✓	✓	✗				
pat:						a	b	x	y	a	b	x	z
iteration-3						✓	✓	✓	✓	✓	✓	✓	✓

In fact we can do even better than just skipping unnecessary alignments. By looking more closely at which comparisons need to be performed within an alignment we can further reduce the number of comparisons we must make.

Imagine that an even smarter algorithm processes the pattern before attempting to perform any matching, that is it *preprocesses* **pat**. During this preprocessing it learns that the substring **pat**$[1 \ldots 3]$ (i.e 'abx') appears

again at **pat**[5 . . . 7]. Following the ninth comparison (the eighth in iteration 2), the algorithm also discovers that,

$$\mathbf{pat}[5 \ldots 7] = \mathbf{txt}[6 \ldots 8].$$

But since the algorithm already knows that,

$$\mathbf{pat}[1 \ldots 3] = \mathbf{pat}[5 \ldots 7],$$

after the shift, when **pat**[1 . . .] is being compared with **txt**[6 . . .], it already knows that,

$$\mathbf{pat}[1 \ldots 3] = \mathbf{txt}[6 \ldots 8],$$

and it can skip comparing the characters in these substrings. This saves us a further 3 comparisons, giving 14 in total.

	1	2	3	4	5	6	7	8	9	10	11	12	13
txt:	x	a	b	x	y	a	b	x	y	a	b	x	z
pat:	a	b	x	y	a	b	x	z					
iteration-1	✗												
pat:		a	b	x	y	a	b	x	z				
iteration-2		✓	✓	✓	✓	✓	✓	✓	✗				
pat:						a	b	x	y	a	b	x	z
iteration-3									✓	✓	✓	✓	✓

In the example above we saw how we could use information about the structure of the pattern and the text to avoid unnecessary comparisons, making our search more efficient. However, we have assumed that this structural information can be gathered efficiently. After all, it is pointless to avoid comparisons if the task of determining which ones we can avoid requires more effort than performing those comparisons in the first place! Fortunately, there are several algorithms that permit us to implement these ideas efficiently. In particular, we will directly examine three algorithms that achieve this by preprocessing the pattern. In other words, they avoid unnecessary comparisons by first exerting a modest amount of effort to learn about the internal structure of the pattern. Other algorithms, like those based on suffix trees, preprocess the text to achieve similar efficiency gains in the exact pattern matching problem, but we will defer the examination of this approach until a later section.

4 Gusfield's Z-Algorithm

There exist several linear time methods for the exact pattern matching problem that preprocess the pattern. However, the respective preprocessing phases, as originally presented, vary significantly in conceptual difficulty. Fortunately, much of the preprocessing required by many of these algorithms (we examine *Boyer Moore's* and the *Knuth-Morris-Pratt* (KMP) algorithm in FIT3155) is remarkably similar. Thus, we will begin our examination of these methods by first describing *Gusfield's Z-Algorithm*, which we find invaluable when attempting to efficiently implement the preprocessing of both Boyer Moore and KMP. In addition, we will see that the Z-Algorithm itself will lead directly to a linear time approach to the exact pattern matching problem.

Even though Gusfield's Z-Algorithm is typically used in the preprocessing phase, it is useful to abstract it from this specific application and describe it with respect to a general string. Put simply, the purpose of the Z-Algorithm is to determine the *Z-values* for a given string in linear time. To make this more concrete let's define exactly what we mean by Z-values,

Definition 2: Z-values

For a string **str**[1 . . . n], define Z_i (for each position $i > 1$ in **str**) as the length of the **longest** substring starting at position i of **str** that matches its prefix (i.e. **str**[i . . . i+Z_i-1] = **str**[1 . . . Z_i]).

The set of values $\{Z_i,$ for $2 \leq i \leq n\}$ for the string **str**[1...n] are what we refer to as its Z-values.

Example 4: Z-values

If we consider, $\mathbf{str}[1\ldots 11] =$ `a a b c a a b x a a y` the corresponding Z-values are,

$$\begin{array}{ccccc} Z_2 = 1 & Z_3 = 0 & Z_4 = 0 & Z_5 = 3 & Z_6 = 1 \\ Z_7 = 0 & Z_8 = 0 & Z_9 = 2 & Z_{10} = 1 & Z_{11} = 0 \end{array}$$

For instance, if we examine Z_5 we see that the substring $\mathbf{str}[5\ldots 7]$ ('aab') matches the prefix $\mathbf{str}[1\ldots 3]$ and that $\mathbf{str}[8] \neq \mathbf{str}[4]$ and so the **longest** substring starting at position 5 in **str** that matches its prefix is $\mathbf{str}[5\ldots 7]$. The length of this substring is 3 and so we obtain $Z_5 = 3$.

Question 1: Does Z_1 matter?

We define Z-values for $2 \leq i \leq n$, but what about Z_1? What would it be for the example above? Is this always the case? How much does this tell us about the structure of the string?

The Z-values of a string define intervals (substrings) that match their prefix. Conceptually we can view each Z-value as defining a box that encapsulates the corresponding substring called a *Z-box*.

Definition 3: Z-boxes

For a string $\mathbf{str}[1\ldots n]$, and for any $i > 1$ such that $Z_i > 0$, the Z_i-box is defined as the interval $[i \ldots i{+}Z_i{-}1]$ of **str**.

Example 5: Z-boxes

Returning to our previous example - $\mathbf{str}[1\ldots 11] =$ `a a b c a a b x a a y` - the Z-boxes would be,

$Z_2 = 1$	Z_2-box $= [2..2]$	$Z_7 = 0$	undefined
$Z_3 = 0$	undefined	$Z_8 = 0$	undefined
$Z_4 = 0$	undefined	$Z_9 = 2$	Z_9-box $= [9..10]$
$Z_5 = 3$	Z_5-box $= [5..7]$	$Z_{10} = 1$	Z_{10}-box $= [10..10]$
$Z_6 = 1$	Z_6-box $= [6..6]$	$Z_{11} = 0$	undefined

To better visualise the Z-boxes we often explicitly draw a box (hence the name) around each of the substrings that match their prefix as shown below.

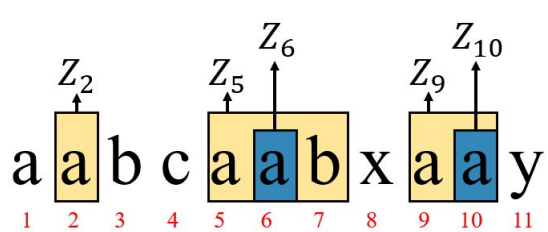

Figure 1: The Z-boxes for the string $\mathbf{str}[1\ldots 11] =$ aabcaabxaay.

We will soon see that to efficiently compute the Z-values for a string we will need to keep track of particular Z-boxes. In preparation for this we introduce a compact method for storing the intervals that define them. Specifically, rather than storing the entire interval $[i \ldots i{+}Z_i{-}1]$ we can simply keep track of the endpoints (i and $i{+}Z_i{-}1$) of the Z-box.

Definition 4: r_i

For a string **str**$[1..n]$, and for all $i > 1$, r_i is the **right-most endpoint** of all Z-boxes that begin at or before position i.

Alternately, r_i is the largest value of j+Z_j-1 over all $1 < j \leq i$, such that $Z_j > 0$.

Example 6: r_i values

If we inspect figure 1 we can see that the r_i values for **str**= 1 2 3 4 5 6 7 8 9 10 11
a a b c a a b x a a y are:

$Z_2 = 1$	$Z_7 = 0$	Z_2-box $= [2..2]$	$r_2 = 2$	$r_7 = 7$
$Z_3 = 0$	$Z_8 = 0$	Z_5-box $= [5..7]$	$r_3 = 2$	$r_8 = 7$
$Z_4 = 0$	$Z_9 = 2$	Z_6-box $= [6..6]$	$r_4 = 2$	$r_9 = 10$
$Z_5 = 3$	$Z_{10} = 1$	Z_9-box $= [9..10]$	$r_5 = 7$	$r_{10} = 10$
$Z_6 = 1$	$Z_{11} = 0$	Z_{10}-box $= [10..10]$	$r_6 = 7$	$r_{11} = 10$

Definition 5: l_i

For a string **str**$[1..n]$, and for all $i > 1$, l_i is the **left end** of the Z-box that ends at r_i.

In the case that there is more than one Z-box ending at r_i, then l_i can be chosen to be the left end of **any** of those Z-boxes.

Example 7: l_i values

Again by examining figure 1 and the corresponding r_i values we can see that the l_i values for
str= 1 2 3 4 5 6 7 8 9 10 11
a a b c a a b x a a y are:

$Z_2 = 1$	$Z_7 = 0$	Z_2-box $= [2..2]$	$l_2 = 2$	$l_7 = 5$
$Z_3 = 0$	$Z_8 = 0$	Z_5-box $= [5..7]$	$l_3 = 2$	$l_8 = 5$
$Z_4 = 0$	$Z_9 = 2$	Z_6-box $= [6..6]$	$l_4 = 2$	$l_9 = 9$
$Z_5 = 3$	$Z_{10} = 1$	Z_9-box $= [9..10]$	$l_5 = 5$	$l_{10} = 9$
$Z_6 = 1$	$Z_{11} = 0$	Z_{10}-box $= [10..10]$	$l_6 = 5$	$l_{11} = 9$

Note that the Z_9-box and the Z_{10}-box both end at $i = 10$ and that $Z_{11} = 0$. Due to this we have $r_{9-11} = 10$ and we have defined $l_{9-11} = 9$. However, it would also be valid to store the left end of the Z_{10}-box for $i \geq 10$ giving $l_{10-11} = 10$.

Example 8: Self study: calculating Z_i, Z_i-box and (l_i, r_i).

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
str= a a b a a b c a x a a b a a b c y

$Z_2 = 1$	$Z_{10} = 7$	Z_2-box $= [2..2]$	$(l_2, r_2) = (2, 2)$	$(l_{10}, r_{10}) = (10, 16)$
$Z_3 = 0$	$Z_{11} = 1$	Z_4-box $= [4..6]$	$(l_3, r_3) = (2, 2)$	$(l_{11}, r_{11}) = (10, 16)$
$Z_4 = 3$	$Z_{12} = 0$	Z_5-box $= [5..5]$	$(l_4, r_4) = (4, 6)$	$(l_{12}, r_{12}) = (10, 16)$
$Z_5 = 1$	$Z_{13} = 3$	Z_8-box $= [8..8]$	$(l_5, r_5) = (4, 6)$	$(l_{13}, r_{13}) = (10, 16)$
$Z_6 = 0$	$Z_{14} = 1$	Z_{10}-box $= [10..16]$	$(l_6, r_6) = (4, 6)$	$(l_{14}, r_{14}) = (10, 16)$
$Z_7 = 0$	$Z_{15} = 0$	Z_{11}-box $= [11..11]$	$(l_7, r_7) = (4, 6)$	$(l_{15}, r_{15}) = (10, 16)$
$Z_8 = 1$	$Z_{16} = 0$	Z_{13}-box $= [13..15]$	$(l_8, r_8) = (8, 8)$	$(l_{16}, r_{16}) = (10, 16)$
$Z_9 = 0$	$Z_{17} = 0$	Z_{14}-box $= [14..14]$	$(l_9, r_9) = (8, 8)$	$(l_{17}, r_{17}) = (10, 16)$

4.1 The Algorithm

Given a string $\mathbf{str}[1 \ldots n]$, the goal of Gusfield's Z-algorithm is to compute $\{Z_i,\ Z_i\text{-box},\ l_i,\ r_i\}$ for each position $1 < i \leq n$ in $O(n)$-time. However, we don't explicitly store all of these values. In particular, we store:

- All successively computed Z_i-values starting from $i = 2$.
  - The Z_i-box can be computed from Z_i in $O(1)$ time and so we do not store it explicitly.
- Variables (l, r), which keep track of the most recently computed (l_i, r_i) values.
  - To compute $(l_i,\ r_i)$ at iteration i we only require the values (l_j, r_j) for $j = i - 1$. At the start of iteration i we have $(l, r) = (l_{i-1}, r_{i-1})$ and these values are then used to update (l, r) so that, $(l, r) = (l_i, r_i)$.

Naive computation of the Z-values for a string would require explicitly matching the characters of each substring $\mathbf{str}[i \ldots]$ to the prefix until a mismatch is found.

Question 2: Naive Z-value computation

What would be the worst case time-complexity for naively computing the Z-values for a string $\mathbf{str}[1 \ldots \mathrm{n}]$?

The Z-algorithm attempts to avoid performing such comparisons where possible by using the information stored in the previously computed Z-boxes. With these ideas in mind we are ready to examine the details of the algorithm.

Base case

To begin, we compute Z_2 by explicit left-to-right comparison of characters $\mathbf{str}[2 \ldots]$ with $\mathbf{str}[1 \ldots]$ until a mismatch is found (recall that Z_2 is the length of the matching substring). We then update the values of l and r accordingly:

If $Z_2 > 0$

- set r (i.e., r_2) to $Z_2 + 1$
- set l (i.e., l_2) to 2

else (i.e., if $Z_2 == 0$)

- set r (i.e., r_2) to 0
- set l (i.e., l_2) to 0

The general case

Let's now imagine that we are attempting to compute Z_k at position k, and assume inductively that:

- We have correctly computed values Z_2 through to Z_{k-1},
- $r = r_{k-1}$ and
- $l = l_{k-1}$.

At this point the following two scenarios can arise:

Case 1: If $k > r$

In this case k is **outside** the right-most Z-box as shown in figure 2, which means that we must again perform explicit comparisons in order to compute Z_k. Specifically, we:

- Compute Z_k by explicitly comparing characters $\mathbf{str}[k \ldots q-1]$ with $\mathbf{str}[1 \ldots q-k]$ until a mismatch is found at some $q \geq k$.
- If $Z_k > 0$:
  - set r (i.e., r_k) to $q - 1$.
  - set l (i.e., l_k) to k.

If $Z_k == 0$ then there is no need to update l and r and they retain the same values as before.

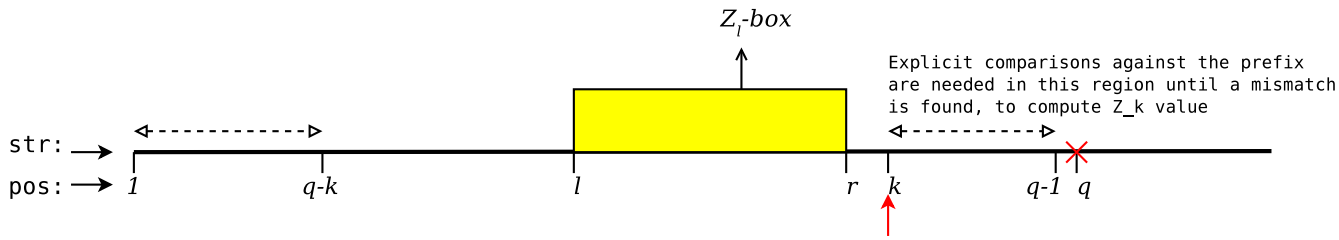

Figure 2: Case 1 of the Z-Algorithm. As $k > r$ the previously computed Z-values cannot be used to inform the computation of Z_k and we must perform explicit comparisons with the prefix until a mismatch is found.

In this case we must compute Z_k through naive comparisons as our previously computed Z-values offer us no insight into the structure of $\mathbf{str}[k \dots n]$. However, if k resides inside a previously computed Z-box this is no longer the case.

Case 2: If $k \leq r$

In this case k is **inside** the right-most Z-box as shown below in figure 3. That is the character $\mathbf{str}[k]$ lies in the substring $\mathbf{str}[l \dots r]$ i.e. within the Z_l-box.

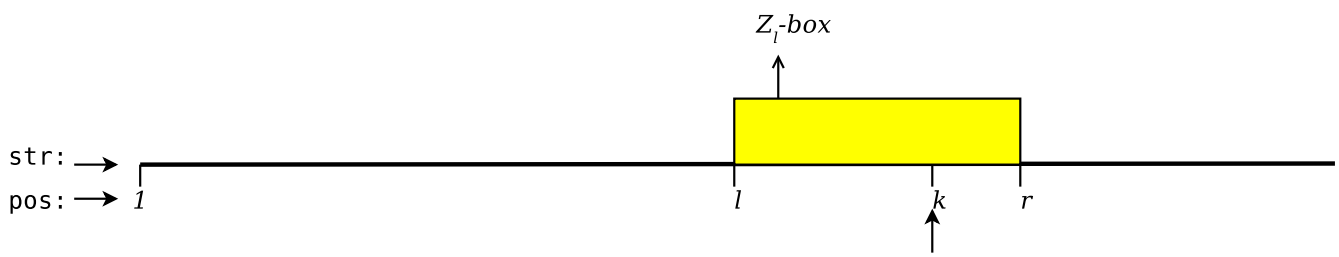

Figure 3: Case 2 of the Z-Algorithm. As $k \leq r$ we can now use previously computed Z-values to inform the computation of Z_k.

By definition of the Z_l-box we know that,

$$\mathbf{str}[1 \dots Z_l] = \mathbf{str}[l \dots r],$$

or that the yellow regions match in figure 4.

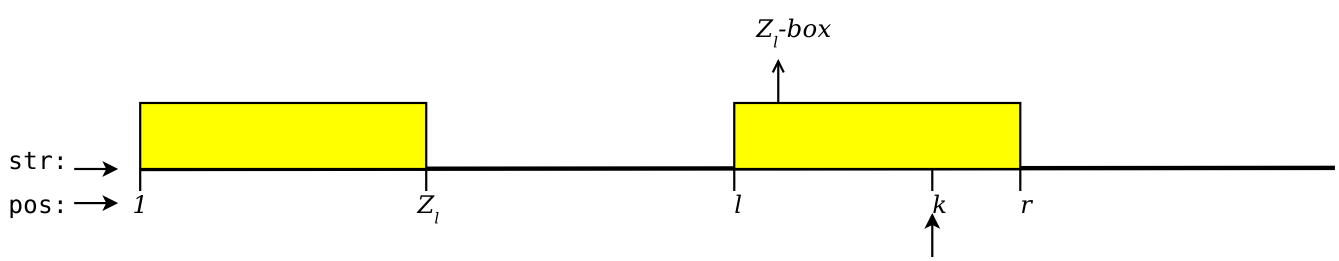

Figure 4: By definition of the Z_l-box the substring $\mathbf{str}[l \dots r]$ must match the prefix $\mathbf{str}[1 \dots Z_l]$. Both regions are depicted by a yellow box.

This then implies that,

$$\mathbf{str}[k] = \mathbf{str}[k - l + 1],$$

as these characters were found to match in the computation of the Z_l-box. By the same logic we can conclude that,

$$\mathbf{str}[k \dots r] = \mathbf{str}[k - l + 1 \dots Z_l],$$

or that the green regions in figure 5 match.

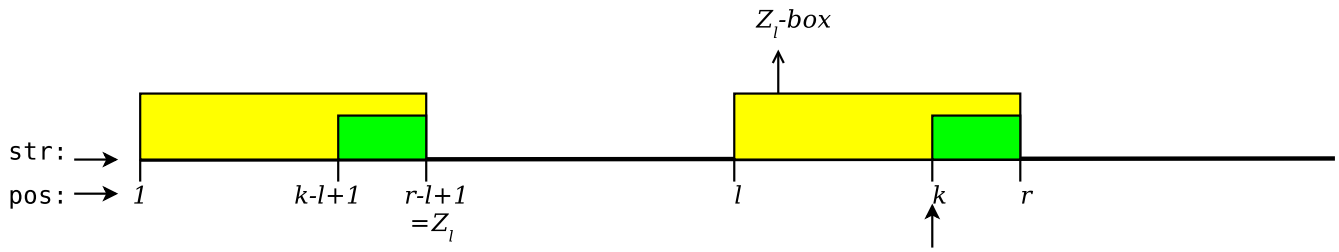

Figure 5: Due to the properties of the Z_l-box the substrings $\mathbf{str}[k-l+1 \ldots Z_l]$ and $\mathbf{str}[k \ldots r]$ must match. These substrings are depicted by the green boxes, while the yellow boxes denote the Z_l-box and its matching prefix.

By our inductive hypothesis we have already correctly computed Z_{k-l+1}, which is shown as a red box in figure 6. We already know that,

$$\mathbf{str}[k \ldots r] = \mathbf{str}[k-l+1 \ldots Z_l],$$

so it is reasonable to expect that the value of Z_{k-l+1} might be able to aid in the computation of Z_k. This is indeed the case and is handled by two separate subcases $2a$ and $2b$.

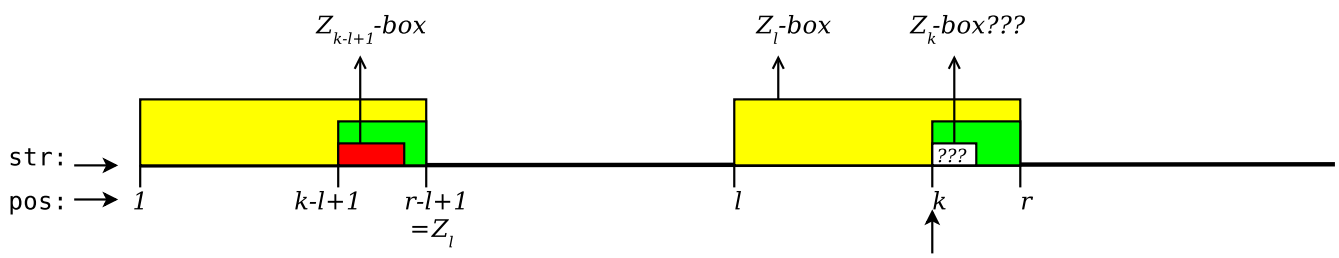

Figure 6: The structure of related Z-boxes in case 2 ($k \leq r$) of the Z-algorithm. The Z_{k-l+1}-box is shown as a red box, while the Z_l-box and its matching prefix are depicted by the yellow boxes, and the matching substrings $\mathbf{str}[k-l+1 \ldots Z_l]$ and $\mathbf{str}[k \ldots r]$ are depicted by the green boxes.

Case $2a$: If $Z_{k-l+1} < r-k+1$ (and $k \leq r$)

- Set Z_k to Z_{k-l+1}
- r and l remain unchanged

In this case the Z_{k-l+1}-box does not extend to the end of the prefix $\mathbf{str}[1 \ldots Z_l]$ that matches the Z_l-box and so there exists some character x at position p, such that,

$$k-l+1 \leq p \leq r-l+1 = Z_l,$$

and

$$\mathbf{str}[p] \neq \mathbf{str}[p-k+l]. \tag{1}$$

That is, there must exist a character - let's call it x - within the green region in figure 7 ($\mathbf{str}[k-l+1 \ldots Z_l]$) that mismatches the corresponding character y ($\mathbf{str}[p-k+l]$) in the prefix $\mathbf{str}[1 \ldots Z_{k-l+1}]$. We know that,

$$\mathbf{str}[k \ldots r] = \mathbf{str}[k-l+1 \ldots Z_l],$$

and by extension,

$$\mathbf{str}[p+l-1] = \mathbf{str}[p].$$

But $\mathbf{str}[p+l-1]$ is compared against y when computing Z_k, and we already know that $x \neq y$! Thus, we can conclude that $Z_k = Z_{k-l+1}$ without making any explicit comparisons.

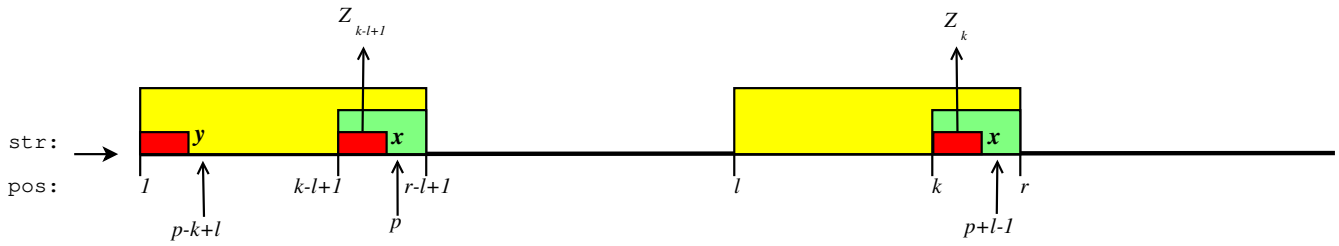

Figure 7: Case 2a of the Z-Algorithm. As $Z_{k-l+1} < r - k + 1$ we can immediately conclude that $Z_k = Z_{k-l+1}$ without performing any explicit comparisons.

Question 3: Case 2a and index wrangling

In the explanation above all of the indices were defined relative to p, the index of the character immediately after the end of the Z_{k-l+1}-box. However, we did not explain explicitly how we arrived at all of the other indices i.e. $p - k + 1$ and $p + l - 1$, or intuit why they are the indices we want. To rectify this compute the Z-values for,

$\mathbf{str}[1 \ldots 13] =$ 1 2 3 4 5 6 7 8 9 10 11 12 13
a b c a b d e a b c a b d and focus in particular on how the Z-Algorithm would compute Z_5 and Z_8. Define the indices l, k, r and p for these cases and sanity check the logic above.

Case 2b: If $Z_{k-l+1} \geq r - k + 1$ (and $k \leq r$)

- Z_k must also be $\geq r - k + 1$
- So, start explicitly comparing $\mathbf{str}[r + 1]$ with $\mathbf{str}[r - k + 2]$ and so on until mismatch occurs.
- Say the mismatch occurred at position $q \geq r + 1$, then:
  - set Z_k to $q - k$,
  - set r to $q - 1$.
  - set l to k.

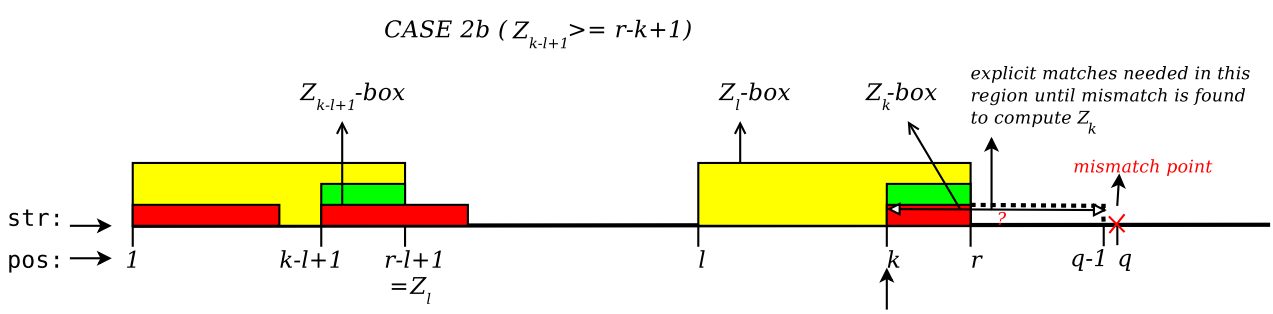

Figure 8: Case 2b of the Z-Algorithm. Explicit comparisons need to be performed starting from $r+1$ in order to compute Z_k

In this case the Z_{k-l+1}-box extends beyond the prefix $\mathbf{str}[1 \ldots Z_l]$ (the first yellow box in figure 8) that we know to match the Z_l-box. We still have that,

$$\mathbf{str}[k \ldots r] = \mathbf{str}[k - l + 1 \ldots Z_l],$$

but does the remainder of the Z_{k-l+1}-box ($\mathbf{str}[r - l + 2 \ldots k - l + Z_{k-l+1}]$) tell us anything about the characters beyond the right end of the Z_l-box ($\mathbf{str}[r + 1 \ldots]$)? Given that we have already decided we must perform explicit comparisons starting from $r + 1$ the answer might seem obvious, but there are some subtleties that warrant a closer examination.

Let's denote the characters at position $r - l + 2$ and $r + 1$, as x and y respectively. Similarly, we will let z denote the character at position $r - k + 2$, which is the character in the prefix $\mathbf{str}[1 \ldots Z_{k-l+1}]$ that is matched against x when computing Z_{k-l+1}. In addition, let's restrict ourselves to the case where Z_{k-l+1} is strictly equal to $r - k + 1$ as shown in figure 9.

We know that x cannot match y otherwise the Z_l-box would be longer. Similarly, we know that x cannot match z otherwise the Z_{k-l+1}-box would be longer. However, for general alphabets[1] this doesn't allow us to infer whether y will

[1] If we had a binary alphabet $x \neq z$ and $x \neq y$ would imply that $y = z$, but this logic cannot be extended to larger alphabets.

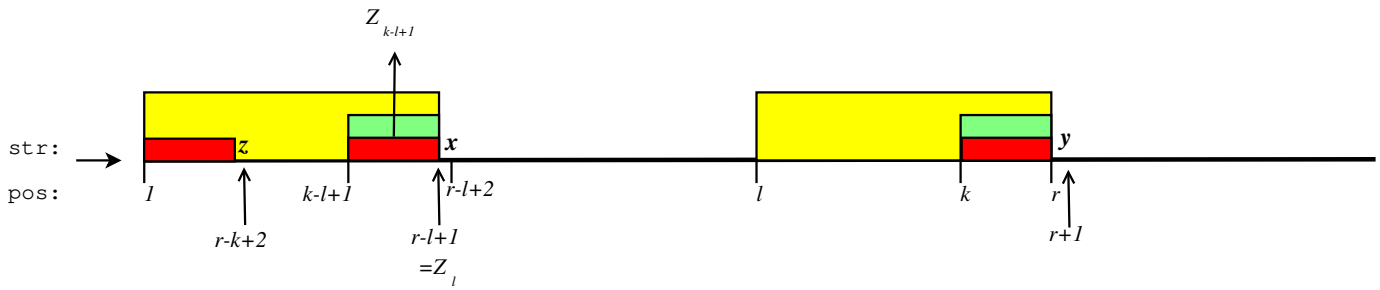

Figure 9: Case 2b of the Z-algorithm, where we have restricted $Z_{k-l+1} = r - k + 1$.

match z or not. Instead we only know that,

$$\mathbf{str}[k \dots r] = \mathbf{str}[k - l + 1 \dots Z_l],$$

which guarantees that the Z_k-box is at least $r-k+1$ characters long, but we must compare characters in the range $\mathbf{str}[r+1 \dots]$ with the corresponding characters in the prefix until a mismatch occurs in order to determine the full extent of the Z_k-box. A similar analysis can be applied to the case where $Z_{k-l+1} > r - k + 1$, and this will be explored in tutorials as the result is somewhat surprising!

Time complexity of Gusfield's Z-Algorithm

Given a string $\mathbf{str}[1 \dots n]$ the Z-Algorithm computes all the Z-values for the string in $O(n)$-time.

Proof:
The total time taken by the algorithm is directly proportional to the sum of:

1. The number of iterations.
2. The number of character comparisons i.e. the total number of matches and mismatches.

The Z-Algorithm computes a single Z-value per iteration giving $n - 1$ iterations. Excluding character comparisons, each iteration involves only constant time operations and so all that remains is to determine the total number of character comparisons made throughout the algorithm.

If we perform any character comparisons during an iteration they stop once the first mismatch is encountered and no further comparisons are made within that iteration. Thus, we can have at most $n - 1$ mismatches. To count the total number of matches we note that $r_k \geq r_{k-1}$ for every iteration k. In every iteration the update to r_k is of the form

$$r_k = r_{k-1} + \delta,$$

where $\delta \geq 0$. However, we also know that $0 \leq r_k \leq n$ (no Z-box can exceed the boundaries of the string) and so there are at most n matches. Therefore, we have $O(n)$ comparisons over all iterations plus $O(n)$ additional work giving an $O(n)$ algorithm overall. ■

4.2 Summary

Given a string $\mathbf{str}[1 \ldots n]$ the Z-Algorithm is an $O(n)$-time algorithm that computes the Z-values of **str** as follows:

Base case

Compute Z_2 by explicit left-to-right comparison of characters $\mathbf{str}[2 \ldots]$ with $\mathbf{str}[1 \ldots]$ until a mismatch is found.

If $Z_2 > 0$

- set r (i.e. r_2) to $Z_2 + 1$
- set l (i.e. l_2) to 2

else (i.e., if $Z_2 == 0$)

- set r (i.e. r_2) to 0
- set l (i.e. l_2) to 0

General case (computing $2 < k \leq n$)

Case 1: If $k > r$

- Compute Z_k by explicitly comparing characters $\mathbf{str}[k \ldots q-1]$ with $\mathbf{str}[1 \ldots q-k]$ until a mismatch is found at some $q \geq k$.
- If $Z_k > 0$:
  - set r (i.e. r_k) to $q - 1$.
  - set l (i.e. l_k) to k.

If $Z_k == 0$ then there is no need to update l and r and they retain the same values as before.

Case 2: If $k \leq r$

- Case $2a$: If $Z_{k-l+1} < r - k + 1$
  - Set Z_k to Z_{k-l+1}
  - r and l remain unchanged
- Case $2b$: If $Z_{k-l+1} \geq r - k + 1$
  - Start explicitly comparing $\mathbf{str}[r+1]$ with $\mathbf{str}[r-k+2]$ and so on until mismatch occurs.
  - Say the mismatch occurred at position $q \geq r + 1$, then:
    - set Z_k to $q - k$,
    - set r to $q - 1$.
    - set l to k.

4.3 Application to the exact pattern matching problem

Although the Z-algorithm is be central to the preprocessing required by other pattern matching algorithms, it can be used directly to obtain a linear time solution to the exact pattern matching problem.

Linear time pattern matching using Gusfield's Z-algorithm

Given a pattern $\mathbf{pat}[1 \ldots m]$ and a reference text $\mathbf{txt}[1 \ldots n]$ we can find all occurrences of **pat** in **txt** using the following method:

- Construct a new string **str** by concatenation as follows:
  $\mathbf{str} = \mathbf{pat}[1 \ldots m] + \$ + \mathbf{txt}[1 \ldots n]$.
  Note, $|\mathbf{str}| = m + 1 + n$.
- Preprocess Z_i values corresponding to **str**, for $1 < i \leq m + n + 1$.
- For any $i > m + 1$, all $Z_i = m$ identifies an occurrence of $\mathbf{pat}[1 \ldots \mathrm{m}]$ at position i in **str**, and hence at position $i - (m + 1)$ in **txt**. That is, $\mathbf{pat}[1 \ldots m] = \mathbf{str}[i \ldots i + m - 1] \equiv \mathbf{txt}[i - (m + 1) \ldots i - 2]$.
- We already, showed that the computation of Z_i values for any sting **str** takes $O(|\mathbf{str}|)$ time.
- Thus, this pattern matching algorithm takes $O(m + n)$ time.

Question 4: Space complexity of pattern matching via the Z-algorithm

How much space do we require to perform pattern matching using the Z-algorithm as detailed above? Can we do any better than this?